Final Project

Designing a System That Monetizes Data

Big Data Systems

Spring 2026 | National Taiwan University

1. Overview

In this course, you have learned how to collect, store, process, and analyze large-scale datasets. Data is often called *"the new oil"*, but raw data on its own has no intrinsic value — value emerges only when data is refined into a product that someone is willing to pay for. The goal of this final project is to bridge the gap between **technical capability** and **economic value**: you will design an end-to-end system that turns data into revenue, and you will defend the business model behind it.

You are not asked to build a unicorn. You are asked to think rigorously about *who would pay, why they would pay, and what it would take technically and operationally to deliver something they would pay for.*

2. Learning Objectives

By completing this project, you will be able to:

- Identify a real, monetizable use case grounded in concrete evidence rather than speculation.
- Design a data pipeline (ingestion → storage → processing → delivery) that supports your value proposition.
- Articulate a customer segment, willingness to pay, and go-to-market plan.
- Anticipate and reason about the practical, legal, and competitive obstacles your product would face in the real world.
- Communicate technical and business decisions clearly in writing and (optionally) in a deployed artifact.

3. The Task

Design a system that monetizes data. You must propose *both* the technical system and the business model that surrounds it. Your submission must clearly address every required item below. Treat each as a section in your final report.

Required Component 1: Target Customer

Who do you want to sell to?
Identify the specific customer segment your system serves. Be precise — "everyone who uses data" is not an answer. A good answer names a role, an industry, a company size, or a concrete user persona, and explains *why* this segment is the right wedge.
Consider:

- Is your customer an individual consumer, a small business, an enterprise, a government

agency, or another data platform (B2B2X)?
- What job are they trying to do today, and what tool or workaround do they currently use?
- Why is your system a better answer than the status quo for this customer?

Required Component 2: Evidence of Demand and Willingness to Pay

How do you know this need exists, and how much would the customer invest (in time, effort, or money)?
This is the most important part of the project. You must show your *data acquisition process* — how you went out and gathered evidence that the problem is real. You may reference and build upon the methodology you used in **Homework 2**.
Your evidence may include, but is not limited to:
- Interviews or surveys with potential users (include the questions you asked and a summary of responses).
- Analysis of public data: job postings, forum discussions, search trends, marketplace listings, app store reviews, government statistics, etc.
- Pricing benchmarks from existing competitors or analogous products.
- Quantified estimates of the customer's *willingness to pay* — either in monetary terms (subscription fee, per-query price, etc.) or in non-monetary terms (time spent, manual labor saved).

Document the full process, not just the conclusion. We want to see how you got from a vague idea to a defensible claim about demand.

Bonus Component 3: Go-to-Market Difficulties (Bonus Points)

What difficulties might you face when promoting this product?
Strong projects acknowledge that building the system is only half the battle. Reflect honestly on the obstacles your product would face. Examples include, but are not limited to:
- **Trust and adoption:** Why would a customer trust your data or your system over an incumbent?
- **Data acquisition costs:** Are the data sources you depend on free, licensable, or scraped? What are the risks?
- **Legal and privacy concerns:** GDPR, PDPA, copyright, terms of service, and platform-specific restrictions.
- **Cold-start problems:** Network effects, chicken-and-egg dynamics, or lack of initial data volume.
- **Competition and moats:** Who else could do this, and what stops them?
- **Unit economics:** At what scale does the system become profitable?

This section is optional but strongly recommended. Thoughtful answers will earn bonus points.

4. System Design Expectations

In addition to the business analysis above, your report must describe the **technical system** that delivers the product. At minimum, address:

- **Data sources:** What data do you ingest, and how? (APIs, scraping, partnerships, user-

generated content, sensors, etc.)

- **Storage and processing:** What technologies do you use, and why are they appropriate for the scale and shape of your data? Reference the tools and paradigms covered in this course (e.g., distributed file systems, batch and stream processing frameworks, NoSQL / SQL stores, message queues).
- **Delivery:** How does the customer consume the product? (Web app, dashboard, API, report, data feed, embedded widget, etc.)
- **Architecture diagram:** Include a clear diagram showing the end-to-end flow.
- **Scalability & cost (optional):** Sketch how the system would behave (and what it would cost) at 10× or 100× your initial scale.

5. Deliverables

You must submit:

1. A **PDF report**. The file must be named `<student_id>.pdf` (for example, `b12902000.pdf`).
2. A **GitHub repository** containing all source code, configuration, sample data (or instructions to obtain it), and a clear `README.md`.
3. The GitHub repository URL must be clearly written on the **first page** of the PDF.

5.1 What the GitHub Repository Should Contain

- All code for ingestion, processing, and delivery.
- A `README.md` explaining how to run the system locally (or how to access the deployed version).
- Instructions or scripts for reproducing any data-collection step you used to establish demand.
- A short architecture overview (can mirror the diagram in your PDF).

5.2 Deployment (Bonus)

Deploying your system earns bonus points. You may deploy on any platform (Vercel, Render, Fly.io, AWS, GCP, your own VM, etc.). If you deploy, include the live URL on the first page of your PDF alongside the GitHub link. The deployment does not need to be production-grade — a working demo is sufficient.

6. Grading Rubric

Criterion	Weight
Clarity and specificity of the target customer (Component 1)	20%
Quality of demand evidence and data-acquisition process (Component 2)	25%
Technical system design and implementation quality (architecture, choice of tools, code, repository organization, reproducibility)	40%
Clarity of writing, diagrams, and overall presentation	15%
Bonus: Thoughtful analysis of go-to-market difficulties	up to +10%
Bonus: Working live deployment	up to +10%

7. Logistics and Rules

- **Language:** The report must be written in **English**.
- **Format:** The report must be submitted as a single PDF file.
- **Originality:** You may reuse code from open-source libraries with proper attribution. Do not copy substantial portions of text or code from other sources without citation.
- **Data ethics:** If your data acquisition involves scraping, surveys, or any third-party data, you are responsible for respecting terms of service, robots.txt, and applicable privacy regulations.

8. Suggested Approach

If you are unsure where to start, here is one possible workflow. You are *not* required to follow it.

1. **Pick a domain you genuinely care about.** Projects fueled by personal curiosity tend to be far more interesting and rigorous than those chosen for convenience.
2. **Find a wedge.** Identify one specific user and one specific problem before you fall in love with a grand vision.
3. **Validate before building (optional).** It is often worth collecting some evidence before investing heavily in implementation — consider talking to potential users, scraping public sources, or studying competitors. If you cannot find anyone who would want the product, you might want to reconsider the product.
4. **Prototype the data flow.** Build the smallest end-to-end pipeline that delivers value, even if it only handles a tiny slice of data.
5. **Scale and harden (optional).** Once the core flow works, you may want to address scalability, fault tolerance, and cost.

6. **Ship the demo (optional).** Consider deploying something — even a read-only dashboard — that a grader can open in a browser.
7. **Write the report last.** Once you know what you actually built and what you learned, the report becomes much easier to write.

A Final Note

The most memorable projects in this course are not the ones with the most sophisticated machine learning models or the largest Spark clusters — they are the ones where the student clearly understood *who* they were building for and *why anyone would pay*. Treat this project as practice for the kind of judgment you will need in industry or in a startup: technical taste and business taste, exercised together.

Good luck, and have fun.